MLFP05 — Deep Learning Textbook

- Module 5 — Deep Learning: Architectures for Vision, Sequence, and Generation
  - Learning Outcomes
  - Prerequisites
  - How to Read This Chapter
- Lesson 5.1: Autoencoders
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: The autoencoder architecture
    - FOUNDATIONS: Four variants
    - THEORY: The VAE ELBO derivation
    - THEORY: The reparameterisation trick
    - ADVANCED: Additional autoencoder variants
  - Mathematical Foundations
    - THEORY: KL divergence between two Gaussians
  - The Kailash Engine: ModelVisualizer (latent space plots)
  - Worked Example: Four Autoencoders on Fashion-MNIST
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.2: CNNs and Computer Vision
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: The convolution operation
    - THEORY: CNN output size formula
    - FOUNDATIONS: Pooling
    - THEORY: ResNet skip connections
    - FOUNDATIONS: SE blocks and modern enhancements
    - ADVANCED: Vision Transformers (ViT)
  - Mathematical Foundations
    - THEORY: Why convolutions detect patterns
    - THEORY: Parameter count comparison
  - The Kailash Engine: OnnxBridge (model export)
  - Worked Example: Building a CNN for Fashion-MNIST
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.3: RNNs and Sequence Models
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: The vanilla RNN
    - THEORY: LSTM — all six gate equations
    - THEORY: Perplexity
    - FOUNDATIONS: Temporal attention
  - The Kailash Engine: ModelVisualizer (training curves)
  - Worked Example: Singapore Stock Price Prediction with LSTM
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.4: Transformers
  - Why This Matters

- Core Concepts
  - THEORY: Self-attention from scratch
  - THEORY: Why divide by $\sqrt{d_k}$
  - THEORY: Multi-head attention
  - THEORY: Positional encoding
  - FOUNDATIONS: Layer normalisation
  - FOUNDATIONS: Transformer variants
- The Kailash Engine: ModelVisualizer (attention visualisation)
- Worked Example: Self-Attention from Scratch and BERT Fine-Tuning
- Try It Yourself
- Cross-References
- Reflection

- Lesson 5.5: Generative Models — GANs and Diffusion
  - Why This Matters
  - Core Concepts
    - THEORY: GAN minimax objective
    - FOUNDATIONS: Mode collapse
    - THEORY: WGAN and gradient penalty
    - FOUNDATIONS: FID (Frechet Inception Distance)
    - ADVANCED: Diffusion models
  - Worked Example: DCGAN and WGAN on Fashion-MNIST
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.6: Graph Neural Networks
  - Why This Matters
  - Core Concepts
    - THEORY: GCN propagation rule
    - FOUNDATIONS: Message passing
    - THEORY: GAT attention weights
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.7: Transfer Learning
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: The transfer learning recipe
    - FOUNDATIONS: Architecture selection guide
    - FOUNDATIONS: ONNX export and InferenceServer
  - Worked Example: Fine-Tuning ResNet for Image Classification
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 5.8: Reinforcement Learning
  - Why This Matters
  - Core Concepts
    - THEORY: Bellman equations
    - THEORY: DQN (Deep Q-Network)
    - THEORY: PPO (Proximal Policy Optimization)
    - FOUNDATIONS: Five algorithms, five applications
  - Worked Example: DQN for Customer Churn Prevention
  - Try It Yourself
  - Cross-References
  - Reflection

- Chapter Summary
  - What Module 6 builds on
- Glossary
- Further Reading

Module 5 — Deep Learning: Architectures for Vision, Sequence, and Generation

> *"Every architecture is a hypothesis about the structure of the world."*

This chapter is where the training toolkit from Lesson 4.8 meets specialised neural architectures. In Module 4 you built a feedforward network from scratch and understood that hidden layers are automated feature engineering with error feedback. Now you will see how different architectures impose different structural biases on that feature learning — biases that make learning dramatically more efficient for specific data types.

A convolutional neural network assumes spatial locality: nearby pixels are more related than distant ones. A recurrent neural network assumes temporal dependency: the meaning of a word depends on the words before it. A transformer assumes that any element can attend to any other element, weighted by relevance. A graph neural network assumes that information flows along edges. Each assumption is a hypothesis about the data's structure, and when the hypothesis is correct, the architecture learns faster and generalises better than a generic feedforward network.

Every architecture in this chapter is implemented in PyTorch. You will write `nn.Module` subclasses, define `forward()` methods, configure `torch.optim` optimisers, and train with gradient descent. The DL training toolkit — dropout, batch normalisation, learning rate scheduling, gradient clipping, early stopping — applies uniformly across all architectures. What changes is the architecture; what remains constant is the training methodology.

By the end of this chapter you will have implemented autoencoders, CNNs, RNNs, transformers, GANs, GNNs, and RL agents. You will know which architecture to use for which data type, how to transfer pre-trained models to new tasks, and how to export models for production deployment.

Learning Outcomes

By the end of this chapter you will be able to:

- Build and train autoencoders (vanilla, denoising, variational, convolutional) and generate new data from VAE latent spaces by deriving the ELBO and reparameterisation trick.
- Implement CNNs with modern enhancements (ResNet skip connections, SE blocks, mixed precision training, Mixup augmentation) and explain the convolution output size formula.
- Build LSTM and GRU networks, write all six LSTM gate equations, apply temporal attention, and train sequence models for time-series prediction and text generation.
- Derive scaled dot-product self-attention from scratch, explain the $\sqrt{d_k}$ normalisation, implement multi-head attention, and fine-tune pre-trained BERT for downstream tasks.
- Implement DCGAN and WGAN with gradient penalty, explain mode collapse and how Wasserstein distance addresses it, and evaluate generative quality with FID.
- Build graph convolutional networks for node and graph classification, implement message passing, and use torch_geometric.
- Fine-tune pre-trained vision and NLP models using transfer learning, export models to ONNX, and deploy with InferenceServer.

- Implement DQN and PPO reinforcement learning algorithms, create custom Gymnasium environments, and explain how RL connects to RLHF for LLM alignment.

Prerequisites

Module 4 complete. Specifically, Lesson 4.8 is non-negotiable — this chapter assumes you can:

- Build a neural network from scratch (forward pass, backprop, gradient descent).
- Implement and explain dropout, batch normalisation, weight initialisation, Adam, and learning rate scheduling.
- Read training curves and diagnose overfitting, underfitting, vanishing gradients, and exploding gradients.
- Explain representation learning: hidden layers discover features guided by a loss function.

PyTorch basics. All code in this chapter uses PyTorch (`torch`, `torch.nn`, `torch.optim`, `torch.utils.data`). If you have not used PyTorch before, spend one hour on the official "60 Minute Blitz" tutorial before starting. The core concepts map directly from Lesson 4.8: `nn.Linear` replaces your manual weight matrices, `nn.ReLU` replaces your `relu()` function, `loss.backward()` replaces your manual backpropagation, and `optimizer.step()` replaces your manual weight update.

Notation:

- $\mathbf{W}^{(l)}$ is the weight matrix for layer l.
- $\odot$ is element-wise (Hadamard) product.
- σ is the sigmoid function unless otherwise noted.
- $\text{softmax}(\mathbf{z})_i = e^{z_i}/\sum_j e^{z_j}$.
- $\mathcal{N}(\mu, \sigma^2)$ is a Gaussian with mean μ and variance σ^2.
- $\text{KL}(q \| p)$ is the Kullback-Leibler divergence from p to q.

How to Read This Chapter

Same structure as all previous modules: Why This Matters, Core Concepts, Mathematical Foundations, Kailash Engine, Worked Example, Try It Yourself (5+ drills), Cross-References, Reflection.

The three-layer depth markers continue:

Marker	Audience	How to Read It
FOUNDATIONS:	Practitioner with M4	Architecture intuition, PyTorch code, practical advice.
THEORY:	Intermediate	Full derivations, loss function analysis, convergence arguments.
ADVANCED:	Masters / researcher	Paper references, frontier results, open problems.

Estimated reading time per lesson:

Lesson	Title	Reading	Exercise	Total
5.1	Autoencoders	110 min	70 min	~3h
5.2	CNNs and Computer Vision	120 min	75 min	~3h 15m

Lesson	Title	Reading	Exercise	Total
5.3	RNNs and Sequence Models	120 min	70 min	~3h 10m
5.4	Transformers	130 min	80 min	~3h 30m
5.5	Generative Models — GANs and Diffusion	120 min	70 min	~3h 10m
5.6	Graph Neural Networks	100 min	60 min	~2h 40m
5.7	Transfer Learning	100 min	65 min	~2h 45m
5.8	Reinforcement Learning	130 min	80 min	~3h 30m

Total: roughly 25 hours. Lesson 5.4 (Transformers) and 5.8 (Reinforcement Learning) are the densest.

Lesson 5.1: Autoencoders

Why This Matters

In Module 4, PCA compressed data into a lower-dimensional linear subspace. The reconstruction was limited to linear combinations of the original features. What if the data lies on a curved manifold — like a Swiss roll or a nonlinear blend of facial features? Linear PCA cannot capture that curvature. An autoencoder can.

An autoencoder is a neural network that learns to reconstruct its input through a bottleneck. The encoder compresses the input to a low-dimensional latent representation. The decoder reconstructs the original input from that representation. By minimising reconstruction error, the network learns a compressed representation that captures the most important features of the data — just like PCA, but non-linear.

The Variational Autoencoder (VAE) goes further: it makes the latent space a probability distribution, enabling you to generate entirely new data by sampling from it. VAEs are foundational to modern generative AI, and the ELBO (Evidence Lower Bound) objective you will derive in this lesson reappears in diffusion models, variational inference, and Bayesian deep learning.

Core Concepts

FOUNDATIONS: The autoencoder architecture

An autoencoder consists of two parts:

Encoder q_ϕ: maps input $\mathbf{x}$ to latent representation $\mathbf{z}$: $\mathbf{z} = q_\phi(\mathbf{x})$

Decoder p_θ: maps latent representation $\mathbf{z}$ back to reconstructed input $\hat{\mathbf{x}}$: $\hat{\mathbf{x}} = p_\theta(\mathbf{z})$

The loss is the reconstruction error:

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

The bottleneck (latent dimension $d_z < d_x$) forces the network to learn a compressed representation. If $d_z \geq d_x$, the network could learn the identity function, which is useless.

FOUNDATIONS: Four variants

Vanilla autoencoder. The simplest form. Encoder and decoder are fully connected layers. Latent dimension is a hyperparameter. The learned representation is deterministic.

```
import torch
import torch.nn as nn

class VanillaAutoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, latent_dim),
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid(),
        )

    def forward(self, x):
        z = self.encoder(x)
        x_hat = self.decoder(z)
        return x_hat
```

Denoising autoencoder (DAE). Corrupt the input by adding noise (Gaussian noise, random zeroing, or salt-and-pepper), then train the network to reconstruct the clean version. This forces the network to learn robust features that are not sensitive to small perturbations.

Convolutional autoencoder. Replace fully connected layers with convolutional layers (encoder) and transposed convolutional layers (decoder). Ideal for image data because convolutions respect spatial locality.

Variational autoencoder (VAE). The encoder outputs parameters of a distribution (mean μ and log-variance $\log \sigma^2$) rather than a deterministic point. The latent representation is sampled from this distribution. This makes the latent space smooth and continuous, enabling generation of new data by sampling.

THEORY: The VAE ELBO derivation

We want to maximise the marginal log-likelihood of the data:

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$$

This integral is intractable (we cannot compute it analytically). Instead, we introduce an approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ and derive a lower bound.

Start with:

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$$

Multiply and divide by $q_\phi(\mathbf{z} \mid \mathbf{x})$:

$$= \log \int q_\phi(\mathbf{z} \mid \mathbf{x}) \frac{p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} \, d\mathbf{z}$$

By Jensen's inequality ($\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$ for concave log):

$$\geq \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log \frac{p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} \, d\mathbf{z}$$

$$= \mathbb{E}_{q_\phi}[\log p(\mathbf{x} \mid \mathbf{z})] - \mathrm{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$$

This is the **ELBO** (Evidence Lower BOund):

$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi}[\log p(\mathbf{x} \mid \mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\text{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))}_{\text{Regularisation term}}$$

The reconstruction term encourages accurate reconstruction. The KL term encourages the approximate posterior to be close to the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, keeping the latent space well-structured.

THEORY: The reparameterisation trick

To backpropagate through the sampling step $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$, we cannot differentiate through a random sample directly. The reparameterisation trick separates the randomness:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Now $\mathbf{z}$ is a deterministic function of $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, and $\boldsymbol{\epsilon}$. Since $\boldsymbol{\epsilon}$ does not depend on the model parameters, gradients flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ as usual. This is what makes VAE training possible with standard backpropagation.

```python
class VAE(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(input_dim, 256), nn.ReLU())
        self.fc_mu = nn.Linear(256, latent_dim)
        self.fc_logvar = nn.Linear(256, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 256), nn.ReLU(),
            nn.Linear(256, input_dim), nn.Sigmoid(),
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterise(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + std * eps

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterise(mu, logvar)
        x_hat = self.decoder(z)
        return x_hat, mu, logvar

def vae_loss(x, x_hat, mu, logvar):
    recon = nn.functional.binary_cross_entropy(x_hat, x, reduction="sum")
```

```
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + kl
```

ADVANCED: Additional autoencoder variants

- **Sparse autoencoder:** adds an L1 penalty on hidden activations, encouraging most neurons to be inactive. Learns sparse, interpretable features.
- **Contractive autoencoder:** adds a penalty on the Frobenius norm of the Jacobian of the encoder, making the representation robust to small input perturbations.
- ***β*-VAE:** scales the KL term by $\beta > 1$ to encourage disentangled latent factors — each latent dimension captures a single factor of variation.

Mathematical Foundations

THEORY: KL divergence between two Gaussians

For the VAE regularisation term with $q = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$ and $p = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\text{KL}(q \| p) = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right)$$

This has a closed-form solution, so no sampling is needed for the KL term — only the reconstruction term requires sampling (via reparameterisation).

The Kailash Engine: ModelVisualizer (latent space plots)

```
from kailash_ml import ModelVisualizer

viz = ModelVisualizer()
fig = viz.scatter(latent_df, x="z1", y="z2", color="digit_label",
                  title="VAE Latent Space — MNIST")
```

Worked Example: Four Autoencoders on Fashion-MNIST

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

transform = transforms.Compose([transforms.ToTensor()])
train_data = datasets.FashionMNIST(root="./data", train=True, download=True,
        transform=transform)
train_loader = DataLoader(train_data, batch_size=128, shuffle=True)

# Train VAE
vae = VAE(input_dim=784, latent_dim=16)
optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)

for epoch in range(20):
    total_loss = 0
    for batch, _ in train_loader:
        batch = batch.view(-1, 784)
        x_hat, mu, logvar = vae(batch)
        loss = vae_loss(batch, x_hat, mu, logvar)
        optimizer.zero_grad()
```

```
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch}: loss = {total_loss / len(train_data):.4f}")

# Generate new images by sampling from the latent space
with torch.no_grad():
    z_sample = torch.randn(16, 16)
    generated = vae.decoder(z_sample).view(-1, 1, 28, 28)
```

Try It Yourself

Drill 1. Implement a vanilla autoencoder with latent dimension 32 and train it on Fashion-MNIST. Compute reconstruction error on the test set. Visualise 10 original images alongside their reconstructions.

Solution:

```
ae = VanillaAutoencoder(784, 32)
optimizer = torch.optim.Adam(ae.parameters(), lr=1e-3)
for epoch in range(20):
    for batch, _ in train_loader:
        batch = batch.view(-1, 784)
        x_hat = ae(batch)
        loss = nn.functional.mse_loss(x_hat, batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Drill 2. Implement a denoising autoencoder. Add Gaussian noise ($\sigma = 0.3$) to the input during training. Compare reconstruction quality with the vanilla autoencoder. Does the DAE produce sharper reconstructions?

Solution:

```
def add_noise(x, sigma=0.3):
    return torch.clamp(x + sigma * torch.randn_like(x), 0, 1)

# During training:
noisy_batch = add_noise(batch)
x_hat = ae(noisy_batch)
loss = nn.functional.mse_loss(x_hat, batch)  # compare with clean input
```

Drill 3. Train a VAE with latent dimension 2. Visualise the 2D latent space, colouring each point by its Fashion-MNIST label. Do the classes separate? Generate images by traversing the latent space in a grid from $(-3, -3)$ to $(3, 3)$.

Solution:

```
vae_2d = VAE(784, 2)
# Train as before, then:
with torch.no_grad():
    for batch, labels in test_loader:
        mu, _ = vae_2d.encode(batch.view(-1, 784))
        # Plot mu[:, 0] vs mu[:, 1], coloured by labels
```

Drill 4. Implement a convolutional autoencoder using `nn.Conv2d` and `nn.ConvTranspose2d`. Compare its reconstruction quality with the fully connected VAE on Fashion-MNIST images.

Solution:

```
class ConvAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding=1), nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1),
        nn.Sigmoid(),
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))
```

Drill 5. Implement the β-VAE variant. Train with $\beta = 1, 4, 10$ and observe the effect on the latent space structure. Higher β should produce more disentangled representations (smoother latent space, more separated clusters) at the cost of worse reconstruction.

Solution:

```
def beta_vae_loss(x, x_hat, mu, logvar, beta=4.0):
    recon = nn.functional.binary_cross_entropy(x_hat, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + beta * kl
```

Cross-References

- **Lesson 4.3** used PCA for linear dimensionality reduction. Autoencoders are the non-linear generalisation.
- **Lesson 4.8** introduced the forward pass, backpropagation, and training toolkit. All of that applies here unchanged.
- **Lesson 5.5** will use the VAE's generative capability alongside GANs and diffusion models.
- **Module 6, Lesson 6.2** will use adapter layers that share the bottleneck structure of autoencoders.

Reflection

You should now be able to:

- Derive the VAE ELBO from the marginal log-likelihood using Jensen's inequality.
- Explain the reparameterisation trick and why it enables gradient flow through sampling.
- Implement all four autoencoder variants in PyTorch.
- Generate new data by sampling from a VAE's latent space.
- Explain why a denoising autoencoder learns more robust features than a vanilla autoencoder.

Lesson 5.2: CNNs and Computer Vision

Why This Matters

A feedforward network treats an image as a flat vector of pixels, ignoring the spatial structure entirely. A pixel in the upper-left corner is no more related to its neighbour than to a pixel in the lower-right corner. This is wasteful — images have strong spatial locality, and the patterns that matter (edges, textures, shapes) are local. A

convolutional neural network exploits this structure by using small filters that slide across the image, detecting local patterns. Early layers learn edges and textures; later layers compose these into objects and scenes. This hierarchical feature learning is why CNNs dominate computer vision.

Core Concepts

FOUNDATIONS: The convolution operation

A convolution applies a small filter (also called a kernel) to every position of the input. The filter has learned weights. At each position, the filter's weights are multiplied element-wise with the input values in that region, and the results are summed to produce a single output value. Sliding the filter across the entire input produces a feature map.

Key parameters:

- **Filter size** ($F \times F$): typically 3×3 or 5×5. Smaller filters detect finer patterns.
- **Stride** (S): how many pixels the filter moves at each step. Stride 1 moves one pixel at a time; stride 2 moves two, halving the output size.
- **Padding** (P): zero-valued pixels added around the input border. "Same" padding preserves the spatial dimensions.

THEORY: CNN output size formula

The output spatial dimension after a convolution is:

$$\text{output} = \frac{W - F + 2P}{S} + 1$$

where W is the input width, F is the filter size, P is the padding, and S is the stride. This formula is essential for designing CNN architectures — you must ensure that spatial dimensions are consistent across layers.

Example: input 28×28 (Fashion-MNIST), filter 3×3, padding 1, stride 1: output $= (28 - 3 + 2)/1 + 1 = 28$. Same padding with stride 1 preserves dimensions.

With stride 2: output $= (28 - 3 + 2)/2 + 1 = 14$. The spatial dimension is halved.

FOUNDATIONS: Pooling

Pooling reduces spatial dimensions by summarising local regions. **Max pooling** takes the maximum value in each window. **Average pooling** takes the mean. Pooling makes the representation more compact and slightly more invariant to small translations of the input.

THEORY: ResNet skip connections

As networks get deeper, gradients vanish — the gradient signal becomes exponentially smaller as it passes through many layers, making early layers nearly impossible to train. ResNet solves this with skip connections:

$$\mathbf{H}(\mathbf{x}) = \mathbf{F}(\mathbf{x}) + \mathbf{x}$$

where $\mathbf{F}(\mathbf{x})$ is the residual function learned by the convolutional layers, and $\mathbf{x}$ is the identity shortcut. The gradient of the skip connection is always 1, providing a highway for gradient flow regardless of depth.

Why this works: instead of learning the full mapping $\mathbf{H}(\mathbf{x})$ directly, the network learns the residual $\mathbf{F}(\mathbf{x}) = \mathbf{H}(\mathbf{x}) - \mathbf{x}$. If the identity mapping is approximately correct, the residual is small and easy to learn. This is why very

deep ResNets (50, 101, 152 layers) can be trained effectively.

```
class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += residual  # skip connection
        return torch.relu(out)
```

FOUNDATIONS: SE blocks and modern enhancements

Squeeze-and-Excitation (SE) blocks recalibrate channel-wise features by learning which channels are important:

$$\mathbf{s} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \text{GAP}(\mathbf{x})))$$

where GAP is Global Average Pooling (squeeze each channel to a scalar), and $\mathbf{W}_1, \mathbf{W}_2$ are small fully connected layers (excitation). The output is the input scaled by $\mathbf{s}$.

Mixed precision training uses FP16 for forward and backward passes (faster, less memory) and FP32 for weight updates (maintains precision). PyTorch provides `torch.cuda.amp` for automatic mixed precision.

Mixup augmentation creates training examples by linearly interpolating between pairs of images and their labels: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$, $\tilde{y} = \lambda y_i + (1-\lambda)y_j$, where $\lambda \sim \text{Beta}(\alpha, \alpha)$. This smooths decision boundaries.

ADVANCED: Vision Transformers (ViT)

Vision Transformers split an image into fixed-size patches (e.g., 16×16), flatten each patch into a vector, add positional embeddings, and feed the sequence of patch embeddings into a transformer encoder. Since 2021, ViTs have matched or exceeded CNN performance on image classification, especially with large-scale pre-training.

Mathematical Foundations

THEORY: Why convolutions detect patterns

A convolution $(\mathbf{x} * \mathbf{w})[i,j] = \sum_{m,n} \mathbf{x}[i+m, j+n] \cdot \mathbf{w}[m,n]$ is a template-matching operation. When the input patch matches the filter, the dot product is large. The filter is learned, so the network discovers which templates (edges, textures, shapes) are useful for the task. Weight sharing (the same filter applied everywhere) dramatically reduces the number of parameters and enforces translation equivariance: a pattern detected in one location will be detected in another.

THEORY: Parameter count comparison

For a 28×28 image:

- Fully connected layer to 256 outputs: $28 \times 28 \times 256 = 200,704$ parameters.

- Convolutional layer with 32 filters of size 3×3: $32 \times 1 \times 3 \times 3 + 32 = 320$ parameters.

Convolutions are 600× more parameter-efficient for this layer, which is why CNNs can be very deep without overfitting.

The Kailash Engine: OnnxBridge (model export)

```
from kailash_ml import OnnxBridge

bridge = OnnxBridge()
bridge.export(model, input_shape=(1, 1, 28, 28), output_path="cnn_classifier.onnx")
```

Worked Example: Building a CNN for Fashion-MNIST

```
class FashionCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
            ResBlock(64),
        )
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(0.3),
            nn.Linear(64, 10),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = FashionCNN()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)
criterion = nn.CrossEntropyLoss()

for epoch in range(30):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
    scheduler.step()
    print(f"Epoch {epoch}: loss={total_loss/total:.4f}, acc={correct/total:.3f}")
```

Try It Yourself

Drill 1. Compute the output size at each layer of the FashionCNN using the formula. Verify by printing tensor shapes during a forward pass.

Solution:

```
x = torch.randn(1, 1, 28, 28)
for layer in model.features:
    x = layer(x)
    print(f"{layer.__class__.__name__}: {x.shape}")
```

Drill 2. Add an SE block after the second convolutional layer. Compare training curves with and without SE blocks. Does the SE block improve final accuracy?

Solution:

```
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.fc = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Linear(channels, channels // reduction), nn.ReLU(),
            nn.Linear(channels // reduction, channels), nn.Sigmoid(),
        )

    def forward(self, x):
        scale = self.fc(x).unsqueeze(-1).unsqueeze(-1)
        return x * scale
```

Drill 3. Implement Mixup augmentation. Train with and without Mixup for 30 epochs. Compare test accuracy and calibration (plot reliability diagrams).

Solution:

```
def mixup(x, y, alpha=0.2):
    lam = torch.distributions.Beta(alpha, alpha).sample()
    idx = torch.randperm(x.size(0))
    x_mix = lam * x + (1 - lam) * x[idx]
    y_a, y_b = y, y[idx]
    return x_mix, y_a, y_b, lam
```

Drill 4. Export the trained model to ONNX and load it back. Verify that predictions match between the PyTorch model and the ONNX model on 100 test samples.

Solution:

```
torch.onnx.export(model, torch.randn(1, 1, 28, 28), "fashion_cnn.onnx")
import onnxruntime as ort
session = ort.InferenceSession("fashion_cnn.onnx")
# Compare predictions
```

Drill 5. Visualise the learned filters of the first convolutional layer. What patterns do they detect (edges, textures, gradients)? Compare filters from a trained model versus a randomly initialised model.

Solution:

```
filters = model.features[0].weight.data
# Plot each filter as a 3x3 grayscale image
```

Cross-References

- **Lesson 4.8** provided the training toolkit (batch norm, dropout, Adam, LR scheduling). All of that is used here.
- **Lesson 5.1** used fully connected autoencoders. Convolutional autoencoders combine this lesson with 5.1.
- **Lesson 5.4** introduces Vision Transformers, an alternative to CNNs for image tasks.
- **Lesson 5.7** applies transfer learning with pre-trained ResNets.

Reflection

You should now be able to:

- Implement a CNN with convolution, pooling, batch normalisation, and skip connections.
- Compute output dimensions using the formula $(W - F + 2P)/S + 1$.
- Explain why ResNet skip connections solve the vanishing gradient problem.
- Apply SE blocks, Mixup, and mixed precision training as modern enhancements.
- Export a model to ONNX for deployment.

Lesson 5.3: RNNs and Sequence Models

Why This Matters

Language is a sequence. Stock prices are a sequence. Musical notes are a sequence. A feedforward network or CNN processes each input independently — it has no memory of previous inputs. A recurrent neural network (RNN) maintains a hidden state that is updated at each time step, allowing it to model dependencies across time. But vanilla RNNs suffer from vanishing gradients when sequences are long. LSTMs solve this with a gating mechanism that controls what information to remember and what to forget.

Core Concepts

FOUNDATIONS: The vanilla RNN

At each time step t, the RNN takes the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, and produces a new hidden state:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

The hidden state $\mathbf{h}_t$ is a summary of all inputs up to time t. For long sequences, the gradient of $\mathbf{h}_T$ with respect to $\mathbf{h}_1$ involves T matrix multiplications, causing the gradient to either vanish (if the eigenvalues of $\mathbf{W}_{hh}$ are less than 1) or explode (if greater than 1).

THEORY: LSTM — all six gate equations

The Long Short-Term Memory network introduces a cell state $\mathbf{C}_t$ — a highway for information that flows through the sequence with only additive modifications, avoiding the vanishing gradient problem.

Forget gate — what to discard from the cell state:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Input gate — what new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

Candidate cell state — what the new information looks like:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

Cell state update — forget old + add new:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Output gate — what to expose from the cell state:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

Hidden state — filtered cell state:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

The cell state $\mathbf{C}_t$ flows through time with only element-wise operations (multiply by forget gate, add input), which means the gradient of $\mathbf{C}_T$ with respect to $\mathbf{C}_1$ is a product of forget-gate values — not a product of weight matrices. Since forget-gate values are between 0 and 1, the gradient is bounded, and information can persist over long sequences.

GRU (Gated Recurrent Unit) simplifies LSTM to two gates (update and reset), using fewer parameters:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

THEORY: Perplexity

Perplexity measures how well a language model predicts a sequence. For a sequence of N words:

$$\text{PP} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i \mid w_1, \ldots, w_{i-1})\right)$$

Lower perplexity means the model is less “perplexed” by the data — it assigns higher probability to the observed words. A perplexity of 1 means perfect prediction; a perplexity of V (vocabulary size) means random guessing.

FOUNDATIONS: Temporal attention

Attention allows the model to focus on specific time steps when making a prediction. For each output step, an attention weight is computed over all input hidden states:

$$\alpha_t = \text{softmax}(\mathbf{h}_t^T \mathbf{H})$$

$$\mathbf{c}_t = \alpha_t \mathbf{H}$$

where $\mathbf{H}$ is the matrix of all hidden states and $\mathbf{c}_t$ is the context vector. This mechanism is the precursor to the full self-attention of transformers (Lesson 5.4).

The Kailash Engine: ModelVisualizer (training curves)

```
from kailash_ml import ModelVisualizer

viz = ModelVisualizer()
fig = viz.line(training_df, x="epoch", y=["train_loss", "val_loss"],
               title="LSTM Training Curves")
```

Worked Example: Singapore Stock Price Prediction with LSTM

```
class StockLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers,
                            batch_first=True, dropout=0.2)
        self.attention = nn.Linear(hidden_dim, 1)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        lstm_out, _ = self.lstm(x)  # (batch, seq_len, hidden_dim)
        # Temporal attention
        attn_weights = torch.softmax(self.attention(lstm_out), dim=1)
        context = (attn_weights * lstm_out).sum(dim=1)
        return self.fc(context)

model = StockLSTM(input_dim=8, hidden_dim=64)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Try It Yourself

Drill 1. Implement a character-level LSTM for text generation. Train on a corpus of Singapore Straits Times headlines. Generate 10 new headlines by sampling from the model's output distribution.

Solution:

```
# Build character vocabulary, convert text to sequences of indices
# Train LSTM to predict next character given previous characters
# Generate by repeatedly sampling and feeding back
```

Drill 2. Compare LSTM and GRU on the stock price prediction task. Which has more parameters? Which converges faster? Which achieves lower test MSE?

Solution:

```
lstm_params = sum(p.numel() for p in lstm_model.parameters())
gru_params = sum(p.numel() for p in gru_model.parameters())
print(f"LSTM: {lstm_params:,}, GRU: {gru_params:,}")
```

Drill 3. Add gradient clipping with max_norm = 1.0. Monitor the gradient norm during training. How often does clipping activate? Does it improve final performance?

Solution:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

Drill 4. Visualise attention weights for a specific prediction. Which time steps does the model attend to most? Do the attention patterns make sense for stock price prediction (e.g., attending to recent days more than distant ones)?

Solution:

```
with torch.no_grad():
    lstm_out, _ = model.lstm(sample_input)
    weights = torch.softmax(model.attention(lstm_out), dim=1).squeeze()
    # Plot weights over time steps
```

Drill 5. Implement a multi-layer LSTM with residual connections between layers. Compare with a standard multi-layer LSTM. Does the residual connection improve training on longer sequences (length 100+)?

Solution:

```
class ResidualLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers):
        super().__init__()
        self.layers = nn.ModuleList()
        for i in range(num_layers):
            dim = input_dim if i == 0 else hidden_dim
            self.layers.append(nn.LSTM(dim, hidden_dim, batch_first=True))

    def forward(self, x):
        for i, lstm in enumerate(self.layers):
            out, _ = lstm(x if i == 0 else h)
            h = out + h if i > 0 and out.shape == h.shape else out
        return h
```

Cross-References

- **Lesson 4.8** introduced backpropagation through layers. BPTT (Backpropagation Through Time) is the same algorithm unrolled through time steps.
- **Lesson 5.2** used CNNs for spatial data. RNNs handle temporal data. The two can be combined (ConvLSTM) for spatiotemporal data.
- **Lesson 5.4** will replace the RNN's sequential processing with parallel self-attention. The attention mechanism you just learned is the conceptual ancestor of the transformer.

Reflection

You should now be able to:

- Write all six LSTM gate equations from memory and explain each gate's role.
- Explain why the cell state highway solves the vanishing gradient problem.
- Compare LSTM and GRU in terms of complexity and performance.
- Implement temporal attention and visualise attention weights.
- Train RNNs for time-series prediction and text generation.

Lesson 5.4: Transformers

Why This Matters

The transformer is the architecture behind GPT, BERT, Claude, and every major language model since 2017. It replaced RNNs for most sequence tasks because it processes all positions in parallel (instead of sequentially) and captures long-range dependencies through attention (instead of hoping information persists through gates).

In this lesson you will derive self-attention from scratch — starting from the question "how should a sequence element decide which other elements to pay attention to?" — and build up to the full transformer architecture. The $\sqrt{d_k}$ normalisation factor, multi-head attention, positional encoding, and the encoder-decoder structure will all be derived from first principles.

Core Concepts

THEORY: Self-attention from scratch

Consider a sequence of n vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ (e.g., word embeddings). We want to compute a new representation for each position that incorporates information from all positions, weighted by relevance.

For each position i, we compute three vectors from $\mathbf{x}_i$:

- **Query** $\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i$ — "what am I looking for?"
- **Key** $\mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i$ — "what do I contain?"
- **Value** $\mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i$ — "what information should I contribute?"

The attention weight from position i to position j is the dot product of query i with key j, normalised:

$$\alpha_{ij} = \text{softmax}_j\left(\frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{d_k}}\right)$$

The output for position i is the weighted sum of all values:

$$\mathbf{o}_i = \sum_{j=1}^{n} \alpha_{ij} \mathbf{v}_j$$

In matrix form:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

THEORY: Why divide by $\sqrt{d_k}$

The dot product $\mathbf{q}^T\mathbf{k}$ is the sum of d_k terms. If the entries of $\mathbf{q}$ and $\mathbf{k}$ have zero mean and unit variance, the dot product has variance d_k (by the properties of independent random variables). For large d_k, the dot products become large in magnitude, which pushes the softmax into saturated regions where the gradients are near zero. Dividing by $\sqrt{d_k}$ normalises the variance of the dot products back to approximately 1, keeping the softmax in its sensitive regime.

Concretely: if $d_k = 512$, the dot products have standard deviation $\sqrt{512} \approx 22.6$. Without scaling, many attention weights would be pushed to near 0 or near 1, and the gradient of softmax in those regions is negligible. With scaling, the standard deviation is reduced to approximately 1, and softmax produces meaningful (non-degenerate) distributions.

THEORY: Multi-head attention

A single attention head captures one type of relationship. Multiple heads capture different types simultaneously:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

where $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q{}^i, \mathbf{K}\mathbf{W}_K{}^i, \mathbf{V}\mathbf{W}_V{}^i)$.

Each head operates in a lower-dimensional subspace (d_k/h per head), so the total computation is the same as a single head with full dimensionality.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        B, L, _ = Q.shape
        Q = self.W_Q(Q).view(B, L, self.n_heads, self.d_k).transpose(1, 2)
        K = self.W_K(K).view(B, -1, self.n_heads, self.d_k).transpose(1, 2)
        V = self.W_V(V).view(B, -1, self.n_heads, self.d_k).transpose(1, 2)

        scores = Q @ K.transpose(-2, -1) / (self.d_k ** 0.5)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))
        attn = torch.softmax(scores, dim=-1)
        out = (attn @ V).transpose(1, 2).contiguous().view(B, L, -1)
        return self.W_O(out)
```

THEORY: Positional encoding

Transformers have no inherent sense of position — unlike RNNs, which process sequentially. Positional encodings are added to the input embeddings to provide position information:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

$$\text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

The sinusoidal encoding allows the model to attend to relative positions (the dot product between positions p and $p + k$ is a function of k only). Learned positional embeddings are an alternative and often perform similarly.

FOUNDATIONS: Layer normalisation

Transformers use layer normalisation instead of batch normalisation because sequence lengths vary within a batch. Layer normalisation normalises across the feature dimension for each individual sample:

$$\hat{z}_i = \frac{z_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

where μ and σ^2 are computed over the feature dimension, not the batch dimension.

FOUNDATIONS: Transformer variants

Model	Architecture	Pre-training	Strength
BERT	Encoder only	Masked language modelling	NLU tasks (classification, NER, QA)
GPT	Decoder only	Next token prediction	Generation, few-shot learning
T5	Encoder-decoder	Text-to-text	Unified framework for all NLP tasks

The Kailash Engine: ModelVisualizer (attention visualisation)

```
from kailash_ml import ModelVisualizer
viz = ModelVisualizer()
fig = viz.heatmap(attention_weights, title="Self-Attention Weights")
```

Worked Example: Self-Attention from Scratch and BERT Fine-Tuning

```
# Part A: Self-attention from scratch
import torch

d_model = 64
seq_len = 10
x = torch.randn(1, seq_len, d_model)  # 1 batch, 10 tokens, 64 dims

W_Q = torch.randn(d_model, d_model) * 0.1
W_K = torch.randn(d_model, d_model) * 0.1
W_V = torch.randn(d_model, d_model) * 0.1

Q = x @ W_Q
K = x @ W_K
V = x @ W_V

d_k = d_model
scores = Q @ K.transpose(-2, -1) / (d_k ** 0.5)
attn_weights = torch.softmax(scores, dim=-1)
output = attn_weights @ V

print(f"Scores shape: {scores.shape}")       # (1, 10, 10)
print(f"Attention shape: {attn_weights.shape}") # (1, 10, 10)
print(f"Output shape: {output.shape}")        # (1, 10, 64)

# Part B: BERT fine-tuning for text classification
from transformers import BertTokenizer, BertForSequenceClassification
import torch.optim as optim

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=6)

# Freeze all layers except the classifier head
for param in model.bert.parameters():
    param.requires_grad = False

optimizer = optim.AdamW(model.classifier.parameters(), lr=2e-5)
```

Try It Yourself

Drill 1. Implement scaled dot-product attention from scratch (no PyTorch modules, just matrix operations). Verify the output shape is correct for batch size 4, sequence length 20, and dimension 128.

Solution:

```
B, L, D = 4, 20, 128
x = torch.randn(B, L, D)
Q = x @ torch.randn(D, D); K = x @ torch.randn(D, D); V = x @ torch.randn(D, D)
scores = Q @ K.transpose(-2, -1) / (D ** 0.5)
attn = torch.softmax(scores, dim=-1)
out = attn @ V
assert out.shape == (B, L, D)
```

Drill 2. Demonstrate the $\sqrt{d_k}$ effect empirically. Compute attention weights with and without scaling for $d_k = 512$. Show that without scaling, the attention distribution is peakier (higher max, lower entropy).

Solution:

```
Q, K = torch.randn(1, 10, 512), torch.randn(1, 10, 512)
scores_unscaled = Q @ K.transpose(-2, -1)
scores_scaled = scores_unscaled / (512 ** 0.5)

attn_unscaled = torch.softmax(scores_unscaled, dim=-1)
attn_scaled = torch.softmax(scores_scaled, dim=-1)

print(f"Unscaled max attn: {attn_unscaled.max():.4f}")
print(f"Scaled max attn: {attn_scaled.max():.4f}")
```

Drill 3. Fine-tune BERT for text classification on a small dataset (e.g., 1000 labelled sentences). Compare accuracy when (a) only the classifier head is trained, (b) the last 2 transformer layers are also unfrozen.

Solution:

```
# (a) Freeze all BERT layers, train classifier only
# (b) Unfreeze last 2 layers:
for param in model.bert.encoder.layer[-2:].parameters():
    param.requires_grad = True
```

Drill 4. Compare the BERT fine-tuned model with the LSTM baseline from Lesson 5.3 on the same text classification task. Report accuracy and training time.

Solution:

```
# Run both models on same train/test split, measure accuracy and wall time
```

Drill 5. Implement sinusoidal positional encoding from scratch. Visualise the encoding matrix as a heatmap. Verify that the dot product between position p and $p + k$ is a function of k only (compute for several values of p and k).

Solution:

```
def sinusoidal_pe(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    pos = torch.arange(0, max_len).unsqueeze(1).float()
    div = torch.exp(torch.arange(0, d_model, 2).float() * (-torch.log(torch.tensor(10000.0))
        / d_model))
    pe[:, 0::2] = torch.sin(pos * div)
```

```
    pe[:, 1::2] = torch.cos(pos * div)
    return pe
```

Cross-References

- **Lesson 5.3** introduced attention as a mechanism on top of RNNs. Self-attention removes the RNN entirely.
- **Module 4, Lesson 4.6** used word embeddings as features. Transformers compute contextualised embeddings — the same word gets different representations depending on context.
- **Lesson 5.7** applies transfer learning with pre-trained transformers (BERT, GPT).
- **Module 6** builds extensively on transformers: LLM fundamentals (6.1), fine-tuning (6.2), and RAG (6.4).

Reflection

You should now be able to:

- Derive scaled dot-product attention from first principles.
- Explain why dividing by $\sqrt{d_k}$ is necessary (prevent softmax saturation).
- Implement multi-head attention in PyTorch.
- Fine-tune BERT for a downstream classification task.
- Compare BERT, GPT, and T5 and know when each is appropriate.

Lesson 5.5: Generative Models — GANs and Diffusion

Why This Matters

Autoencoders (Lesson 5.1) generate data by sampling from a latent space, but the samples are often blurry. GANs produce sharper, more realistic outputs through adversarial training — a generator tries to fool a discriminator that tries to distinguish real from fake. The tension between these two networks drives the generator to produce increasingly realistic data.

Core Concepts

THEORY: GAN minimax objective

The GAN training objective is a minimax game:

$$\min_G \max_D [\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z}[\log (1 - D(G(\mathbf{z})))]]$$

The discriminator D maximises the objective by correctly classifying real data as real ($D(\mathbf{x}) \to 1$) and generated data as fake ($D(G(\mathbf{z})) \to 0$). The generator G minimises the objective by producing data that the discriminator classifies as real ($D(G(\mathbf{z})) \to 1$).

At the Nash equilibrium, the generator produces data indistinguishable from real data, and the discriminator outputs 0.5 for everything. In practice, training oscillates and rarely reaches the true equilibrium.

FOUNDATIONS: Mode collapse

Mode collapse occurs when the generator learns to produce only a few types of outputs that fool the discriminator, ignoring the full diversity of the training data. For instance, a GAN trained on MNIST might generate only the digit 1 — the discriminator cannot tell these apart from real 1s, but the generator has stopped producing any other digit.

THEORY: WGAN and gradient penalty

The Wasserstein GAN (WGAN) replaces the JS divergence (implicit in the original GAN) with the Wasserstein (Earth Mover's) distance:

$$\min_G \max_{D \in \text{1-Lip}} [\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[D(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p_z}[D(G(\mathbf{z}))]]$$

where the discriminator (now called a critic) must be 1-Lipschitz. The Wasserstein distance provides a meaningful gradient even when the distributions do not overlap, which is why WGAN training is more stable.

Gradient penalty enforces the Lipschitz constraint by penalising the gradient norm of the critic:

$$\mathcal{L}_{\text{GP}} = \lambda \, \mathbb{E}_{\hat{\mathbf{x}}}[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2]$$

where $\hat{\mathbf{x}}$ is a random interpolation between a real and a generated sample.

```
class WGAN_GP(nn.Module):
    def gradient_penalty(self, real, fake, critic):
        alpha = torch.rand(real.size(0), 1, 1, 1, device=real.device)
        interp = (alpha * real + (1 - alpha) * fake).requires_grad_(True)
        d_interp = critic(interp)
        gradients = torch.autograd.grad(
            outputs=d_interp, inputs=interp,
            grad_outputs=torch.ones_like(d_interp),
            create_graph=True,
        )[0]
        grad_norm = gradients.view(gradients.size(0), -1).norm(2, dim=1)
        return ((grad_norm - 1) ** 2).mean()
```

FOUNDATIONS: FID (Frechet Inception Distance)

FID measures the quality and diversity of generated images by comparing the distribution of real and generated image features (extracted by a pre-trained InceptionV3 network):

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g)^{1/2})$$

Lower FID means the generated distribution is closer to the real distribution. FID captures both quality (mean) and diversity (covariance).

ADVANCED: Diffusion models

Diffusion models (DDPM — Denoising Diffusion Probabilistic Models) add noise to data progressively over T steps, then learn to reverse the process:

- **Forward process:** gradually add Gaussian noise until the data is pure noise.
- **Reverse process:** a neural network learns to denoise step by step.

Diffusion models produce higher-quality and more diverse samples than GANs, at the cost of slower generation (requires many denoising steps). Stable Diffusion and DALL-E are based on diffusion models.

Worked Example: DCGAN and WGAN on Fashion-MNIST

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, 256), nn.BatchNorm1d(256), nn.ReLU(),
            nn.Linear(256, 512), nn.BatchNorm1d(512), nn.ReLU(),
            nn.Linear(512, 784), nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z).view(-1, 1, 28, 28)

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 512), nn.LeakyReLU(0.2),
            nn.Linear(512, 256), nn.LeakyReLU(0.2),
            nn.Linear(256, 1), nn.Sigmoid(),
        )

    def forward(self, x):
        return self.net(x)
```

Try It Yourself

Drill 1. Implement the full DCGAN training loop with alternating generator and discriminator updates. Train for 50 epochs and visualise generated images at epochs 1, 10, 25, and 50.

Solution:

```
G = Generator(); D = Discriminator()
opt_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
criterion = nn.BCELoss()

for epoch in range(50):
    for real, _ in train_loader:
        # Train D
        z = torch.randn(real.size(0), 100)
        fake = G(z).detach()
        loss_D = criterion(D(real), torch.ones(real.size(0), 1)) + \
                 criterion(D(fake), torch.zeros(real.size(0), 1))
        opt_D.zero_grad(); loss_D.backward(); opt_D.step()

        # Train G
        z = torch.randn(real.size(0), 100)
        fake = G(z)
        loss_G = criterion(D(fake), torch.ones(real.size(0), 1))
        opt_G.zero_grad(); loss_G.backward(); opt_G.step()
```

Drill 2. Implement WGAN with gradient penalty. Compare training stability with the original DCGAN (plot discriminator and generator losses over epochs).

Solution:

```
# WGAN critic loss: D(real).mean() - D(fake).mean() + gp
# WGAN generator loss: -D(fake).mean()
```

Drill 3. Compute FID between generated and real Fashion-MNIST images. How does FID change over training epochs?

Solution:

```
from pytorch_fid import fid_score
# Save real and generated images to directories
# fid = fid_score.calculate_fid_given_paths([real_dir, gen_dir], batch_size=64,
        device="cpu", dims=2048)
```

Drill 4. Implement conditional generation: given a class label, generate an image of that class. Modify the generator to take both z and a one-hot class label as input.

Solution:

```
class ConditionalGenerator(nn.Module):
    def __init__(self, latent_dim=100, n_classes=10):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim + n_classes, 256), nn.ReLU(),
            nn.Linear(256, 784), nn.Tanh(),
        )

    def forward(self, z, label_onehot):
        return self.net(torch.cat([z, label_onehot], dim=1)).view(-1, 1, 28, 28)
```

Drill 5. Create a comparison table: VAE vs DCGAN vs WGAN. For each, report training stability, sample quality (FID), sample diversity, and training time. Which would you use for synthetic data generation in a production setting?

Solution: WGAN-GP offers the best balance of stability and quality. VAE produces more diverse but blurrier samples. DCGAN is fast but prone to mode collapse. For production synthetic data, WGAN-GP or diffusion models are preferred.

Cross-References

- **Lesson 5.1** introduced VAEs for generation. GANs produce sharper samples; VAEs produce more diverse samples.
- **Module 6, Lesson 6.3** uses preference alignment (DPO) — a different approach to steering generative models.

Reflection

You should now be able to:

- Write the GAN minimax objective and explain the generator-discriminator dynamic.
- Implement DCGAN and WGAN with gradient penalty.
- Explain mode collapse and how Wasserstein distance addresses it.
- Evaluate generative quality with FID.
- Compare VAE, GAN, and diffusion models for different generation tasks.

Lesson 5.6: Graph Neural Networks

Why This Matters

Social networks, molecular structures, supply chains, and knowledge graphs are naturally represented as graphs — nodes connected by edges. Standard neural networks cannot process graph-structured data directly. GNNs operate by message passing: each node aggregates information from its neighbours, then updates its representation. After several rounds of message passing, each node's representation captures information from its local neighbourhood.

Core Concepts

THEORY: GCN propagation rule

The Graph Convolutional Network (GCN) layer updates node features using the normalised adjacency matrix:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)}\right)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ (adjacency matrix with self-loops), $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$ (degree matrix), $\mathbf{H}^{(l)}$ is the feature matrix at layer l, and $\mathbf{W}^{(l)}$ is the learnable weight matrix.

The normalisation $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ ensures that the aggregated features are averaged (not summed), preventing high-degree nodes from dominating.

FOUNDATIONS: Message passing

The GCN propagation can be viewed as message passing:

1. **Message:** each node sends its current representation to all neighbours.
2. **Aggregate:** each node averages the messages from its neighbours (and itself, via the self-loop).
3. **Update:** the aggregated message is transformed by a linear layer and activation function.

After L layers of message passing, each node's representation captures information from nodes up to L hops away.

THEORY: GAT attention weights

Graph Attention Networks (GAT) compute attention weights between neighbours:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j])$$

$$\alpha_{ij} = \text{softmax}_j(e_{ij})$$

$$\mathbf{h}_i^{'} = \sigma(\Sigma_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j)$$

This allows the model to learn which neighbours are more important for each node.

```
from torch_geometric.nn import GCNConv, GATConv, global_mean_pool
```

```python
class GCN(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.fc = nn.Linear(hidden_channels, out_channels)

    def forward(self, x, edge_index, batch):
        x = torch.relu(self.conv1(x, edge_index))
        x = torch.relu(self.conv2(x, edge_index))
        x = global_mean_pool(x, batch)  # graph-level readout
        return self.fc(x)
```

Try It Yourself

Drill 1. Build a GCN for graph classification on TUDataset (MUTAG or PROTEINS). Report accuracy.

Drill 2. Compare GCN vs GAT on the same dataset. Does attention improve accuracy?

Drill 3. Visualise learned node embeddings after 2 layers of GCN. Do nodes of the same class cluster together?

Drill 4. Vary the number of GCN layers from 1 to 6. Does over-smoothing occur (all node embeddings become similar)?

Drill 5. Implement a simple message-passing network from scratch (without torch_geometric). Verify it produces the same output as GCNConv.

Cross-References

- **Lesson 4.1** introduced spectral clustering, which uses the graph Laplacian — the same matrix that appears in GCN.
- **Lesson 5.4** introduced attention. GAT applies attention to graph neighbours.

Reflection

You should now be able to build GCNs and GATs, explain message passing, and use torch_geometric for graph ML.

Lesson 5.7: Transfer Learning

Why This Matters

Training a model from scratch on a small dataset often leads to overfitting. Transfer learning solves this by starting from a model pre-trained on a large dataset (ImageNet for vision, Wikipedia/BookCorpus for NLP) and fine-tuning it on your small target dataset. The pre-trained model has already learned general features (edges, textures for vision; grammar, semantics for NLP) that transfer to new tasks.

Core Concepts

FOUNDATIONS: The transfer learning recipe

1. **Load a pre-trained model** (e.g., ResNet-50 from ImageNet, BERT from BookCorpus).
2. **Replace the classification head** with a new one matching your number of classes.
3. **Freeze early layers** — they contain general features that transfer well.
4. **Fine-tune later layers** and the new head on your target dataset.
5. **Optionally unfreeze more layers** if you have enough data.

FOUNDATIONS: Architecture selection guide

Data Type	Best Architecture	When to Transfer
Images	CNN / ViT	Always (ImageNet pre-trained)
Text	Transformer	Always (BERT/GPT pre-trained)
Sequences	LSTM / Transformer	Sometimes (domain-specific)
Graphs	GNN	Rarely (task-specific)
Tabular	Gradient boosting	Never (train from scratch)

FOUNDATIONS: ONNX export and InferenceServer

```
from kailash_ml import OnnxBridge, InferenceServer

bridge = OnnxBridge()
bridge.export(model, input_shape=(1, 3, 224, 224), output_path="model.onnx")

server = InferenceServer(model_path="model.onnx")
result = server.predict(sample_input)
batch_results = server.predict_batch(sample_batch)
```

Worked Example: Fine-Tuning ResNet for Image Classification

```
from torchvision import models

model = models.resnet18(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace classifier head
model.fc = nn.Linear(512, 10)  # 10 classes

optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# After a few epochs, optionally unfreeze layer4:
for param in model.layer4.parameters():
    param.requires_grad = True
optimizer = torch.optim.Adam([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3},
])
```

Try It Yourself

Drill 1. Fine-tune ResNet-18 on Fashion-MNIST. Compare accuracy with the CNN from Lesson 5.2. How many epochs does transfer learning need to match the from-scratch accuracy?

Drill 2. Fine-tune BERT for sentiment classification. Compare with the LSTM from Lesson 5.3.

Drill 3. Export both fine-tuned models to ONNX. Measure inference latency.

Drill 4. Implement progressive unfreezing: start with only the head, then unfreeze one layer at a time every 5 epochs. Does this improve final accuracy?

Drill 5. Apply data augmentation (random crop, horizontal flip, colour jitter) to the image dataset. How much does augmentation improve transfer learning performance?

Cross-References

- **Lesson 5.2** built CNNs from scratch. Transfer learning reuses pre-trained CNNs.
- **Lesson 5.4** introduced BERT. Transfer learning with BERT is the practical application.
- **Module 6, Lesson 6.2** extends transfer learning to LoRA and adapter-based fine-tuning.

Reflection

You should now be able to fine-tune pre-trained models for new tasks, export to ONNX, and deploy with InferenceServer.

Lesson 5.8: Reinforcement Learning

Why This Matters

All deep learning so far learns from static datasets — images, text, sequences. Reinforcement learning (RL) learns from interaction with an environment. An agent takes actions, receives rewards, and learns a policy that maximises cumulative reward. RL powers game-playing AI (AlphaGo), robotics, and — crucially — the alignment of large language models (RLHF, which you will study in Module 6).

Core Concepts

THEORY: Bellman equations

The value of a state is the expected cumulative reward from that state:

$$V(s) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s]$$

The value of a state-action pair:

$$Q(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \mid S_t = s, A_t = a]$$

where $\gamma \in [0,1)$ is the discount factor — future rewards are worth less than immediate ones.

THEORY: DQN (Deep Q-Network)

DQN approximates $Q(s, a)$ with a neural network $Q(s, a; \theta)$. The loss is:

$$\mathcal{L} = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

where θ^- is a target network (periodically copied from θ) that stabilises training. DQN handles discrete action spaces.

THEORY: PPO (Proximal Policy Optimization)

PPO is a policy gradient method with a clipped objective that prevents large policy updates:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}}(a_t \mid s_t)$ is the probability ratio, $\hat{A}_t$ is the advantage estimate, and ϵ (typically 0.2) controls how far the new policy can deviate from the old one.

PPO handles continuous action spaces and is the algorithm used in RLHF (Reinforcement Learning from Human Feedback) for LLM alignment.

FOUNDATIONS: Five algorithms, five applications

Algorithm	Action Space	Application
DQN	Discrete	Customer churn prevention
DDPG	Continuous	Manufacturing control
SAC	Continuous	Dynamic pricing
A2C	Discrete/Continuous	Resource allocation
PPO	Discrete/Continuous	Supply chain optimisation

```
import gymnasium as gym

class ChurnEnv(gym.Env):
    """Custom environment for customer churn prevention."""
    def __init__(self):
        super().__init__()
        self.observation_space = gym.spaces.Box(low=0, high=1, shape=(10,))
        self.action_space = gym.spaces.Discrete(3)  # no action, discount, call

    def step(self, action):
        # Compute next state, reward based on action effectiveness
        reward = self._compute_reward(action)
        return next_state, reward, done, False, {}

    def reset(self, seed=None):
        return self._initial_state(), {}
```

Worked Example: DQN for Customer Churn Prevention

```
import torch
import torch.nn as nn

class DQN(nn.Module):
    def __init__(self, state_dim, action_dim):
```

```
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, action_dim),
        )

    def forward(self, x):
        return self.net(x)

# Training loop with experience replay
from collections import deque
import random

replay_buffer = deque(maxlen=10000)

def train_dqn(env, model, target_model, optimizer, episodes=500):
    for episode in range(episodes):
        state, _ = env.reset()
        total_reward = 0
        done = False

        while not done:
            # Epsilon-greedy action selection
            if random.random() < max(0.01, 1.0 - episode / 200):
                action = env.action_space.sample()
            else:
                with torch.no_grad():
                    q_values = model(torch.FloatTensor(state))
                    action = q_values.argmax().item()

            next_state, reward, done, _, _ = env.step(action)
            replay_buffer.append((state, action, reward, next_state, done))
            state = next_state
            total_reward += reward

            # Sample mini-batch and update
            if len(replay_buffer) >= 64:
                batch = random.sample(replay_buffer, 64)
                # Compute DQN loss and update
```

Try It Yourself

Drill 1. Implement DQN with experience replay on a CartPole environment. How many episodes until convergence?

Drill 2. Implement PPO for a continuous control task (e.g., Pendulum-v1). Verify the clipped objective prevents large updates.

Drill 3. Create a custom Gymnasium environment for Singapore taxi pricing. Define state (time, location, demand), actions (price multiplier), and reward (revenue minus customer loss).

Drill 4. Compare DQN and PPO on the same discrete-action environment. Which converges faster? Which achieves higher final reward?

Drill 5. Explain in a paragraph how PPO connects to RLHF for LLM alignment. What is the "environment"? What is the "reward"? What is the "policy"? (This connects to Module 6, Lesson 6.3.)

Solution: In RLHF, the LLM is the policy — it takes a prompt (state) and generates a response (action). The reward comes from a reward model trained on human preferences: responses preferred by humans get higher reward. PPO updates the LLM's weights to increase the probability of generating responses that score highly, while the clipping objective prevents the model from deviating too far from its pre-trained behaviour. Module 6, Lesson 6.3 will show how DPO achieves the same goal without the reward model.

Cross-References

- **Lesson 4.8** introduced gradient descent and loss functions. RL uses the same optimisation but with rewards instead of labels.
- **Module 6, Lesson 6.3** uses DPO and GRPO as alternatives to RLHF, bypassing the reward model.

Reflection

You should now be able to:

- Write the Bellman equations and explain what they represent.
- Implement DQN with experience replay.
- Explain PPO's clipped objective and why it stabilises training.
- Create custom Gymnasium environments.
- Articulate the PPO-to-RLHF connection.

Chapter Summary

Module 5 covered every major deep learning architecture. You built eight types of neural networks, each exploiting a different structural assumption about the data:

Architecture	Assumption	Data Type	Lesson
Autoencoder	Compression	Any (unsupervised)	5.1
CNN	Spatial locality	Images, grids	5.2
RNN/LSTM	Temporal dependency	Sequences, time series	5.3
Transformer	Any-to-any attention	Sequences, text, images	5.4
GAN	Adversarial competition	Generation	5.5
GNN	Graph structure	Networks, molecules	5.6
Transfer Learning	Reuse	Small datasets	5.7
RL	Interaction	Decision-making	5.8

The common thread: every architecture learns features from data via backpropagation, using the DL training toolkit from Lesson 4.8. What changes is the structural bias — convolutions for spatial patterns, recurrence for temporal patterns, attention for relevance patterns, message passing for graph patterns.

What Module 6 builds on

Module 6 is the capstone. It assumes you can:

- Fine-tune pre-trained models (BERT, ResNet) for new tasks.
- Implement and train any architecture from this chapter.
- Export models for deployment with ONNX.

- Explain how RL connects to LLM alignment.

Module 6 will take you from trained models to production LLM applications: prompt engineering, fine-tuning with LoRA, preference alignment with DPO, RAG systems, AI agents with ReAct, multi-agent orchestration, AI governance with PACT, and full production deployment with Nexus.

Glossary

Attention. A mechanism where each element in a sequence computes a weighted combination of all other elements, with weights based on relevance.

Autoencoder. A neural network that learns compressed representations by reconstructing its input through a bottleneck.

Backpropagation Through Time (BPTT). The application of backpropagation to recurrent networks by unrolling the computation graph through time steps.

Batch normalisation. Normalising layer inputs within each mini-batch to stabilise training.

Bellman equation. A recursive equation defining the value of a state as the immediate reward plus the discounted value of the next state.

BERT. Bidirectional Encoder Representations from Transformers. A pre-trained transformer encoder for NLU tasks.

Cell state. The memory component of an LSTM that flows through time with only additive modifications.

CNN. Convolutional Neural Network. Processes grid-structured data using learned filters.

Convolution. A template-matching operation that slides a filter across an input, producing a feature map.

Cosine annealing. A learning rate schedule that follows a cosine curve.

DCGAN. Deep Convolutional GAN. Uses convolutional layers in both generator and discriminator.

Decoder. The component of an autoencoder or transformer that maps from latent space to output space.

Diffusion model. A generative model that learns to reverse a gradual noising process.

Discount factor. The weight γ applied to future rewards in RL, controlling how much the agent values long-term versus immediate rewards.

DQN. Deep Q-Network. Approximates the Q-function with a neural network for discrete action RL.

ELBO. Evidence Lower BOund. The objective function for VAE training, consisting of a reconstruction term and a KL divergence term.

Encoder. The component that maps from input space to latent space.

Experience replay. Storing past transitions in a buffer and sampling from them for training, decorrelating sequential samples.

Feature map. The output of a convolutional filter applied to an input.

FID. Frechet Inception Distance. A metric for evaluating generated image quality and diversity.

Filter. A small learnable matrix used in convolution to detect local patterns.

Fine-tuning. Adapting a pre-trained model to a new task by training on task-specific data.

Forget gate. The LSTM gate that controls what information to discard from the cell state.

GAN. Generative Adversarial Network. A generator and discriminator trained adversarially.

GAT. Graph Attention Network. A GNN that uses attention weights between neighbours.

GCN. Graph Convolutional Network. A GNN that aggregates neighbour features using the normalised adjacency matrix.

GELU. Gaussian Error Linear Unit. Activation function used in transformers.

GPT. Generative Pre-trained Transformer. An autoregressive decoder for text generation.

Gradient penalty. A regularisation term in WGAN that enforces the Lipschitz constraint on the critic.

GRU. Gated Recurrent Unit. A simplified RNN with update and reset gates.

Hidden state. The internal memory of an RNN at each time step.

InferenceServer. Kailash ML engine for serving model predictions.

Input gate. The LSTM gate that controls what new information to store.

Key. One of the three projections (Q, K, V) in attention, representing what each element contains.

Latent space. The low-dimensional space learned by an autoencoder or VAE.

Layer normalisation. Normalising across the feature dimension for each sample, used in transformers.

LSTM. Long Short-Term Memory. An RNN variant with gating mechanisms that prevent vanishing gradients.

Message passing. The GNN mechanism where nodes exchange information along edges.

Mixed precision. Using FP16 for computation and FP32 for weight updates.

Mixup. Data augmentation by linearly interpolating training examples.

Mode collapse. A GAN failure where the generator produces only a limited variety of outputs.

Multi-head attention. Running multiple attention operations in parallel with different projections.

OnnxBridge. Kailash ML engine for exporting models to ONNX format.

Output gate. The LSTM gate that controls what to expose from the cell state.

Perplexity. A measure of language model quality; lower is better.

Policy. In RL, a mapping from states to actions.

Positional encoding. Sinusoidal or learned embeddings added to transformer inputs to provide position information.

PPO. Proximal Policy Optimization. An RL algorithm with clipped objectives for stable training.

Q-function. The expected cumulative reward for taking action a in state s and following the policy thereafter.

Query. One of the three projections (Q, K, V) in attention, representing what each element is looking for.

Reparameterisation trick. Expressing a random sample as a deterministic function of the mean, variance, and independent noise, enabling gradient flow.

Residual connection. A skip connection $H(x) = F(x) + x$ that facilitates gradient flow in deep networks.

ResNet. Residual Network. A CNN architecture with skip connections enabling very deep networks.

Reward. In RL, the scalar feedback signal the agent receives after taking an action.

RLHF. Reinforcement Learning from Human Feedback. Using RL to align LLMs with human preferences.

RNN. Recurrent Neural Network. Processes sequences by maintaining a hidden state across time steps.

SE block. Squeeze-and-Excitation block. Channel recalibration mechanism for CNNs.

Self-attention. Attention where queries, keys, and values all come from the same sequence.

Skip connection. A shortcut that adds the input of a layer directly to its output.

Stride. The step size of a convolutional filter as it slides across the input.

Transfer learning. Reusing a pre-trained model's features for a new task.

Transformer. An architecture based entirely on attention, processing all positions in parallel.

Transposed convolution. An upsampling operation used in decoders to increase spatial dimensions.

Value. One of the three projections (Q, K, V) in attention, representing the information to contribute.

Value function. In RL, the expected cumulative reward from a state.

VAE. Variational Autoencoder. An autoencoder with a probabilistic latent space, enabling generation.

ViT. Vision Transformer. Applies transformer architecture to image patches.

Wasserstein distance. A distance metric between probability distributions used in WGAN.

WGAN. Wasserstein GAN. Uses Wasserstein distance for more stable training.

Further Reading

On autoencoders and VAEs

- Kingma, D., and Welling, M. "Auto-Encoding Variational Bayes." *ICLR*, 2014. The original VAE paper.
- Doersch, C. "Tutorial on Variational Autoencoders." *arXiv:1606.05908*, 2016.

On CNNs

- He, K., et al. “Deep Residual Learning for Image Recognition.” *CVPR*, 2016. The ResNet paper.
- Hu, J., Shen, L., and Sun, G. “Squeeze-and-Excitation Networks.” *CVPR*, 2018. The SE block paper.
- Dosovitskiy, A., et al. “An Image is Worth 16x16 Words.” *ICLR*, 2021. The Vision Transformer paper.

On RNNs and LSTMs

- Hochreiter, S., and Schmidhuber, J. “Long Short-Term Memory.” *Neural Computation*, 1997. The original LSTM paper.
- Cho, K., et al. “Learning Phrase Representations using RNN Encoder-Decoder.” *EMNLP*, 2014. The GRU paper.

On transformers

- Vaswani, A., et al. “Attention Is All You Need.” *NeurIPS*, 2017. The original transformer paper.
- Devlin, J., et al. “BERT: Pre-training of Deep Bidirectional Transformers.” *NAACL*, 2019.
- Radford, A., et al. “Language Models are Unsupervised Multitask Learners.” OpenAI, 2019. The GPT-2 paper.

On GANs and generative models

- Goodfellow, I., et al. “Generative Adversarial Nets.” *NeurIPS*, 2014. The original GAN paper.
- Arjovsky, M., Chintala, S., and Bottou, L. “Wasserstein GAN.” *ICML*, 2017.
- Gulrajani, I., et al. “Improved Training of Wasserstein GANs.” *NeurIPS*, 2017. WGAN-GP.
- Ho, J., Jain, A., and Abbeel, P. “Denoising Diffusion Probabilistic Models.” *NeurIPS*, 2020.

On GNNs

- Kipf, T., and Welling, M. “Semi-Supervised Classification with Graph Convolutional Networks.” *ICLR*, 2017.
- Hamilton, W., Ying, R., and Leskovec, J. “Inductive Representation Learning on Large Graphs.” *NeurIPS*, 2017. GraphSAGE.
- Velickovic, P., et al. “Graph Attention Networks.” *ICLR*, 2018. GAT.

On reinforcement learning

- Sutton, R., and Barto, A. *Reinforcement Learning: An Introduction*. MIT Press, 2018. The definitive textbook. Free online at `incompleteideas.net/book/the-book.html`.
- Mnih, V., et al. “Human-level control through deep reinforcement learning.” *Nature*, 2015. DQN.
- Schulman, J., et al. “Proximal Policy Optimization Algorithms.” *arXiv:1707.06347*, 2017.

On transfer learning

- Zhuang, F., et al. “A Comprehensive Survey on Transfer Learning.” *Proceedings of the IEEE*, 2020.